

Esercizio A1

Data la forma della ricorrenza, è possibile utilizzare il Master Theorem, versione base.

- Per $a = 1$, abbiamo $\alpha = \log_2 1 = 0$, $\beta = 0$; siamo nel secondo caso, $T(n) = \Theta(\log n)$.
- Per $a = 2$, abbiamo $\alpha = \log_2 2 = 1$, $\beta = 1$; siamo nel secondo caso, $T(n) = \Theta(n \log n)$.
- Per $a = 3$, abbiamo $\alpha = \log_2 3 < 2$, $\beta = 2$; siamo nel terzo caso, $T(n) = \Theta(n^2)$.
- In generale, è possibile dimostrare che $\log_2 a < a - 1$ per tutti i valori interi $a \geq 3$; siamo nel terzo caso e si ottiene $T(n) = \Theta(n^{a-1})$.

Esercizio A2

È possibile risolvere questo tipo di problemi con un meccanismo di "sliding window" ed un insieme implementato tramite tabella hash. L'insieme viene utilizzato per contenere i precedenti k valori, per verificare la presenza di duplicati. Una volta che l'insieme contiene i primi $k + 1$ valori (ovvero tutti gli elementi a distanza $\leq k$ rispetto all'elemento in posizione $k + 1$), nei passi successivi si dovrà rimuovere l'elemento a distanza $k + 1$ e verificare che i nuovi elementi inseriti non siano già presenti. Se si arriva in fondo senza incontrare duplicati, si ritorna **false**.

```
boolean closeDuplicates(int[] A, int n, int k)
```

```
    SET S = Set()
    for i = 1 to n do
        if i > k + 1 then                                % Remove the element which is k position away
            S.remove(A[i - (k + 1)])
        if S.contains(A[i]) then % Check if the element is already present in the last k - 1 elements
            return true
        else
            S.insert(A[i])
    return false                                         % No duplicate found
```

Il costo computazionale della soluzione proposta è $\Theta(n)$, assumendo che le operazioni sull'insieme siano eseguibili in $O(1)$. È possibile pensare a soluzioni in tempo $O(nk)$ (due cicli annidati), $O(n \log n)$ (creando delle coppie (valore, indice originale) e ordinando per valore e in caso di parità per indice, per poi controllare la distanza fra valori uguali) e infine $O(n \log k)$ (implementazione dell'insieme basato alberi red-black).

Esercizio A3

Un sottoalbero è simmetrico se i suoi sottoalberi destro e sinistro sono speculari. Due sottoalberi t_1 , t_2 sono speculari se il sottoalbero sinistro di t_1 è speculare al sottoalbero destro di t_2 e il sottoalbero destro di t_1 è speculare al sottoalbero sinistro di t_2 . Il caso base è dato due nodi **nil** (si ritorna **true**) o da uno nodo **nil** e un nodo non **nil** (si ritorna **false**).

La procedura effettua una visita su entrambi gli alberi, e quindi ha complessità $\Theta(n)$.

```
boolean isSymmetric(TREE T)
```

```
    return isMirror(T.left, T.right)
```

```

boolean isMirror(TREE  $t_L$ , TREE  $t_R$ )
  if  $t_L == \text{nil}$  and  $t_R == \text{nil}$  then
    | return true
  else if  $t_L \neq \text{nil}$  and  $t_R \neq \text{nil}$  then
    | return isMirror( $t_L.\text{right}$ ,  $t_R.\text{left}$ ) and isMirror( $t_L.\text{left}$ ,  $t_R.\text{right}$ )
  else
    | return false

```

Errori tipici

- È difficile identificare un approccio seguito da molti, ma direi che ho visto molti tentativi sbagliati basati sul tentativo di contare il numero di foglie "destre" e "sinistre" nei sottoalberi destro e sinistro, e confrontarli in maniera incrociata. Sfortunatamente, è possibile che gli alberi rispettino questo controllo e non siano simmetrici nei nodi interni.

Esercizio B1

L'esercizio può essere risolto estendendo l'approccio per LCS a due stringhe, tenendo conto di tre stringhe.

Sia $DP[i][j][k]$ la lunghezza della longest common subsequence dei prefissi $X(i)$, $Y(j)$, $Z(k)$. Il suo valore può essere calcolato ricorsivamente come segue:

$$DP[i][j][k] = \begin{cases} 0 & i = 0 \vee j = 0 \vee k = 0 \\ DP[i-1][j-1][k-1] + 1 & i > 0 \wedge j > 0 \wedge k > 0 \wedge X[i] = Y[j] = Z[k] \\ \max\{DP[i-1][j][k], DP[i][j-1][k], DP[i][j][k-1]\} & \text{altrimenti} \end{cases}$$

In altre parole, nel caso l'ultimo carattere di tutte e tre le stringhe sia uguale, sia conta un carattere in comune e si "arretra" su tutte le stringhe. Altrimenti, si seleziona il massimo fra eliminare un carattere da una delle stringhe e tenere le altre inalterate.

```

int lcs(ITEM[]  $X$ , ITEM[]  $Y$ , ITEM[]  $Z$ , int  $n_x$ , int  $n_y$ , int  $n_z$ )
  int[][]  $DP = \text{new int}[0 \dots n_x][0 \dots n_y][0 \dots n_z] = \{ 0 \}$ 
  for  $i = 1$  to  $n_x$  do
    | for  $j = 1$  to  $n_y$  do
      | | for  $k = 1$  to  $n_z$  do
        | | | if  $X[i] == Y[j] == Z[k]$  then
          | | | |  $DP[i][j][k] = DP[i-1][j-1][k-1] + 1$ 
        | | | else
          | | | |  $DP[i][j][k] = \max(DP[i-1][j][k], DP[i][j-1][k], DP[i][j][k-1])$ 
      | |
    |
  return  $DP[n_x][n_y][n_z]$ 

```

Il costo computazionale è $\Theta(n_x \cdot n_y \cdot n_z)$, per via dei tre cicli annidati.

Errori tipici

- Molti hanno inizializzato la matrice con dei cicli che scorrevano la tabella DP in questo modo: $DP[i][0][0]$ (con due indici a zero). In realtà, bisogna scrivere tre cicli annidati, ognuno dei quali ha un indice pari a zero e due indici scorrono per tutti i valori possibili. Per semplificare, pur mantenendo il costo computazionale pari a $\Theta(n_x \cdot n_y \cdot n_z)$, ho preferito inizializzare a zero l'intera matrice.